

Localizing Vulnerabilities under Function Inlining Toward Precise Binary Patching

Anonymous Authors

Abstract—The time to weaponize disclosed vulnerabilities has collapsed from years to days, while AI-driven pipelines now discover and validate exploitable bugs faster than remediation can respond, making the patch-compile-test-deploy cycle the critical bottleneck. Precise binary patching can narrow this exposure window by repairing only the vulnerable code in a running binary, but it first requires accurately locating where the vulnerable code resides. Compiler function inlining complicates this task: a vulnerable callee can be absorbed into a larger inliner or fragmented across non-contiguous regions, no longer surviving as an independent function. Existing binary code similarity detection (BCSD) and vulnerability localization approaches operate at function granularity and therefore miss such inlined vulnerabilities. We present PinPoint, a system to be able to localize known vulnerabilities under such function inlining, which is backbone-agnostic over token-based BCSD models. It employs a threshold-gated three-stage cascade using a different window size: whole function, basic block, and token. To this end, we introduce a four-type taxonomy according to inlining relationships. **MJ:** Our evaluation demonstrates the effectiveness and efficiency of PinPoint on 73 CVEs across nine open-source projects using three BCSD backbones. With BinShot, PinPoint improves Top-1 retrieval from 26.9% to 65.5% when a vulnerable function is inlined into a non-vulnerable function. Among queries whose ground-truth target function is retrieved at Top-1, PinPoint-BinShot localizes the embedded vulnerable code in 77.3% of queries where a vulnerable function is inlined into a non-vulnerable function and 94.9% of queries where a vulnerable function is inlined into another vulnerable function. Size-ratio pruning further provides a $2.5\times$ speedup with little change in aggregate retrieval accuracy.

I. INTRODUCTION

According to Zero Day Clock [1], the time required to weaponize a vulnerability (*i.e.*, to develop an in-the-wild exploit from a disclosed CVE [2]) has plummeted from years in 2018 to only days at the time of writing. Traditionally, the interval between vulnerability discovery and patch availability was considered manageable (typically spanning days to weeks) because vulnerability discovery itself was slow, labor-intensive, and constrained by the capacity of humans. However, the emergence of agentic AI systems capable of autonomously discovering software vulnerabilities at scale fundamentally alters this economic balance. In particular, AI-driven pipelines can triage, reproduce, and validate exploitable vulnerabilities at a pace that significantly outpaces human-driven remediation workflows, compressing the effective exposure window to hours or less, turning the traditional patch-compile-test-deploy cycle a critical operational bottleneck.

One promising direction is precise binary patching, which can address this bottleneck directly. Rather than waiting for a source-level fix to propagate through a complete build

and release pipeline, a narrowly scoped binary patch can be generated and deployed to a running system in near real-time, mitigating the vulnerability before adversaries can weaponize it. Beyond speed, precision is equally critical in this context: an autonomous system that produces overly coarse patches risks destabilizing production systems at the same scale and speed at which it secures them, whereas incomplete patches may leave exploitable code paths intact while creating a false sense of protection. As AI-assisted vulnerability discovery and exploit generation become increasingly commoditized, the ability to produce correct, minimal, and immediately deployable binary patches is becoming essential to practical software security.

Precise patching requires accurately locating a known vulnerability in the target binary because a binary-level repair must modify only the vulnerable code rather than wait for a source-level patch to propagate through the full release pipeline. Yet, this precision is challenging: an overly coarse patch may perturb unrelated logic and destabilize production systems, while an incomplete patch may leave exploitable paths intact. However, compiler function inlining (at -O1 and above) further complicates this task by ① losing a function boundary; ② being fragmented into non-contiguous regions; or ③ becoming interleaved with unrelated co-inliners, making precise localization substantially harder. Existing approaches focus on either binary code similarity detection used for vulnerability discovery [3], [4], [5], [6], [7], [8], [9], [10], [11], [12], function-level vulnerability localization [13], [14], [15], or patch presence detection [16], [17], [18], [19], [20], [21], [22], [23], [24], implicitly assuming that the vulnerable code still survives as an independent function.

In this work, we present PinPoint, a system capable of localizing (known) vulnerabilities under function inlining to support precise binary patching. To this end, we introduce a four-type taxonomy of post-inlining binary functions (Figure 1), distinguishing whether a vulnerable function preserves its boundary and whether it inlines or is inlined into other functions. Rather than assuming the vulnerable code remains a standalone routine, PinPoint adapts its search according to where the taxonomy indicates the vulnerable code may reside. By design, PinPoint is a backbone-agnostic layer built on token-based BCSD models that **MJ:** combines size-ratio pruning with a threshold-gated three-stage cascade. In essence, **MJ:** when the vulnerable function remains standalone without participating in inlining, PinPoint begins at Stage 1, which performs whole-function matching, preserving the behavior of existing BCSD models. **MJ:** Inlining-related cases, in which the vulnerable function participates in inlining as an inli-

nee, an inliner, or both, instead begin directly at Stage 2 with basic-block-stride search. From the selected entry stage, when no confident match is found, PinPoint progressively escalates into the candidate function using the next finer-grained search. MJ: At Stages 2 and 3, sliding windows are re-embedded and compared against the vulnerable reference, allowing PinPoint to localize vulnerable inlinees. We evaluate PinPoint on 73 CVEs spanning nine open-source projects, covering 596 vulnerability-relevant functions among 224,188 recovered functions. Our evaluation includes three token-based BCSD backbones (BinShot [4], SAFE [3], and Trex [5]) and two graph-based baselines (VulHawk [12] and Hermes-Sim [11]). When vulnerable code preserves a recoverable boundary, whole-function matching works well. However, the challenge arises when the vulnerable function is absorbed into a non-vulnerable host and no longer survives as an independent routine. PinPoint addresses this limitation through its sliding-window stages, improving Top-1 retrieval accuracy from MJ: 26.9% to 65.5% when the vulnerable function is absorbed into a non-vulnerable host. Among queries whose ground-truth target function is retrieved at Top-1, PinPoint-BinShot localizes the embedded vulnerable code in 77.3% of queries where the vulnerable function is absorbed into a non-vulnerable host and in 94.9% of queries where a vulnerable function also contains an inlined vulnerable function. Size-ratio pruning further provides a $2.5\times$ speedup while having little effect on aggregate retrieval accuracy. Case studies further demonstrate that PinPoint can localize vulnerable code fragmented across non-contiguous regions and nested through multiple inlining steps.

The main contributions of our paper are as follows:

- We present PinPoint, the first vulnerability localization approach that handles compiler-induced function inlining.
- We introduce a four-type taxonomy of functions under compiler inlining.
- We design and implement the prototype of PinPoint, a backbone-agnostic MJ: matching pipeline that combines size-ratio pruning with a threshold-gated three-stage cascade.
- We evaluate PinPoint on 73 CVEs across nine open-source projects with three BCSD backbones, demonstrating its effectiveness and efficiency with varying case studies.

We plan to release our implementation to foster further research¹ in the domain of vulnerability localization.

II. BACKGROUND

Function Inlining and Its Effects. Function inlining is a pervasive compiler optimization in which the body of a callee is substituted directly into its call site, eliminating call overhead and enabling further optimizations, such as constant propagation and dead-code elimination across previously distinct function boundaries. Modern compilers apply inlining aggressively at moderate-to-high optimization levels (e.g., -O1 and above) [25]. As a result, all symbolic traces of the

callee, including its name, address range, and logical boundary, may disappear entirely in stripped executables. These effects propagate to every derived program representation: ① the caller’s control-flow graph contains the basic blocks of the inlined callee; ② the call graph loses the corresponding call edge, and ③ data-flow analyses merge what were originally separate function scopes. Moreover, optimizations such as basic-block reordering and cold-path splitting can scatter an inlined body across non-contiguous address ranges, so the callee’s footprint is not even guaranteed to be a single contiguous region. Under nested inlining, such distortions accumulate recursively, rendering it exceedingly difficult to attribute individual instructions to their originating source functions. To the best of our knowledge, our work first focuses on identifying a vulnerability in the presence of such (nested) inlining functions.

Binary Patching and Precision. Binary patching modifies a compiled executable to remediate a vulnerability in the absence of source code. Existing approaches generally fall into two categories: ① cold patching, which rewrites bytes or redirects control flow via trampolines using frameworks such as LIEF [26] or RetroWrite [27] and ② hot patching, which updates a live process without requiring a restart, as demanded by high-availability systems. In both settings, patch precision is a first-order concern. A patch applied too broadly at the caller level may introduce regressions in unrelated, non-vulnerable logic, whereas a patch applied too narrowly to a single call site may fail to protect other inlined instances of the same vulnerable callee.

III. VULNERABILITY LOCALIZATION UNDER INLINING

A. Problem Formulation

Preliminaries. Given an executable consisting of n binary functions, suppose that at least one function is vulnerable. A vulnerable (binary) function corresponds to a source-level function that has been directly patched to address a disclosed CVE. The left side of Figure 1 illustrates a set of binary functions ($\mathcal{F}$) obtained when the executable is compiled without optimization (e.g., -O0). Let F^{NV} and F^V denote the disjoint sets of non-vulnerable and vulnerable functions, respectively, where $F^{NV} \cap F^V = \emptyset$ and $\mathcal{F} = F^{NV} \cup F^V$. After compiler optimizations are applied on the right side of Figure 1, functions may disappear due to the compiler’s function inlining decisions [25], resulting in the optimized function set $\mathcal{F}_O$. The number of binary functions may decrease, i.e., $|\mathcal{F}| \geq |\mathcal{F}_O|$.

Formal Notations of Function Inlining. With compiler optimizations enabled (e.g., -O[1-3]), both a vulnerable function ($f^V \in F^V$) and a non-vulnerable function ($f^{NV} \in F^{NV}$) may serve as either an inliner or an inlinee, thereby forming an inlining relationship. Furthermore, the inlining may occur either directly or transitively through nested inlining phases. The direct inlining relation is defined as $E \subseteq \mathcal{F} \times \mathcal{F}$, where $(f_i, f_j) \in E$ indicates that function f_i directly inlines function f_j . Using graph notation, direct inlining is represented as

¹<https://anonymous.4open.science/r/pinpoint>

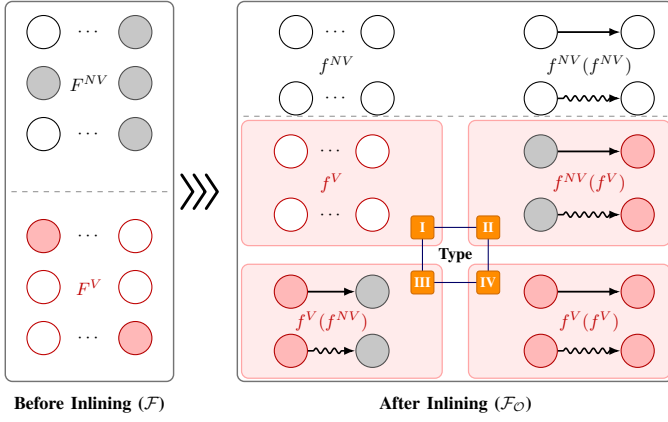


Fig. 1: Taxonomy of binary functions under function inlining. Left (before inlining): a set of functions ($\mathcal{F}$) are partitioned into the disjoint sets F^{NV} and F^V . Right (after inlining): a set of optimized functions ($\mathcal{F}_O$), with the four vulnerability-related cases (square boxes in red) labeled **Type I** f^V , a standalone vulnerable function that retains its function-level boundary; **Type II** $f^{NV}(f^V)$; **Type III** $f^V(f^{NV})$; and **Type IV** $f^V(f^V)$, where $f_i(f_j)$ denotes the inlining relationship with the vulnerability presence of the inliner (outer) and that of any reachable inlee (inner). Note that a light-colored circle represents a function involved in an inlining relationship. In the case of inlining, the top and bottom row of each panel depict direct inlining ($f_i \rightarrow f_j \in E$) and nested inlining ($f_i \rightsquigarrow f_j \in E^+$). Note that Types II-IV are the primary targets of this work, since f^V no longer survives as an independent function (Section III-A).

$f_i \rightarrow f_j$. For instance, $f_i^{NV} \rightarrow f_j^V$ denotes that a non-vulnerable function directly inlines a vulnerable function. Similarly, $f_i^V \rightarrow f_j^{NV}$ represents a vulnerable function directly inlining a non-vulnerable function. Nested (or transitive) inlining is modeled using the transitive closure of E , denoted by E^+ . Formally, $(f_i, f_j) \in E^+$ indicates that f_i transitively inlines f_j through one or more intermediate inlining steps. With graph notation, nested inlining is represented as $f_i \rightsquigarrow f_j$. For example, $f_i^{NV} \rightsquigarrow f_j^V$ means that a non-vulnerable function transitively inlines a vulnerable function through one or more intermediate functions. Likewise, $f_i^V \rightsquigarrow f_j^{NV}$ represents a vulnerable function transitively inlining a non-vulnerable function. Finally, we define an inlining relationship as $f_i(f_j)$ if and only if $\exists e \in E$ or $\exists e \in E^+$.

Vulnerable Functions and Inlining Types. Without inlining, the distinction between F^V and F^{NV} is straightforward. However, compiler inlining optimization inherently propagates vulnerabilities across inliner-inlee relationships, thereby increasing the number of functions that become vulnerability-related after optimization. Accordingly, we classify post-inlining vulnerabilities into the following four categories as illustrated in Figure 1: ① [Type I: f^V] standalone vulnerable function(s) without inlining; ② [Type II: $f^{NV}(f^V)$] non-vulnerable function(s) that inline one or more vulnerable

functions; ③ [Type III: $f^V(f^{NV})$] vulnerable function(s) that inline one or more non-vulnerable functions; and ④ [Type IV: $f^V(f^V)$] vulnerable function(s) that inline one or more vulnerable functions. In our corpus, Type II, III, and IV account for 63.93%, 29.84%, and 6.23%, respectively, among all direct and nested inlining cases.

Problem Statement. Given a known vulnerability (a disclosed CVE) and a target binary B to be inspected, an analyst (e.g., reversing expert or practitioner) aims to determine whether the vulnerable code is present and, if so, to locate it within B with high probability (e.g., Top- K). **MJ:** When the vulnerable function remains a standalone routine without participating in inlining (Type I), function-level retrieval is generally sufficient. For the inlining-related cases (Types II–IV), inlining may either remove the vulnerable function boundary or alter the body of a surviving vulnerable function. We therefore localize the vulnerable code within the recovered target function and report its matched address range $[a, b]$.

B. Challenges and Scope

Challenges. Localizing a vulnerability under function inlining is particularly challenging for several reasons. First, inlining removes or obscures the original function boundaries that existing analysis techniques heavily rely on. Once a vulnerable function is inlined, it no longer exists as a distinct symbol in the binary. The resulting loss of explicit call structure significantly complicates downstream analyses, including taint analysis, symbolic execution, fuzzing coverage, and ML-based vulnerability detection. Second, (aggressive) inlining degrades even debugging and provenance information through instruction reordering, variable merging, stack-frame elimination, source-line folding. As a result, stack traces become less informative; call graph loses edges; and function-level attribution becomes ambiguous. Third, inlining duplicates vulnerable regions because a single source-level vulnerability may map to many distinct binary locations. Inserting the same vulnerable logic at multiple call sites effectively scatters N separate copies of the vulnerable code throughout the binary. Consequently, vulnerability signatures become inconsistent, complicating further analysis tasks, such as binary diffing, patch analysis, and root-cause localization. Lastly, inlining exposes only low-level operations while removing higher-level semantic context, such as API-level intent, security-sensitive abstraction boundaries, or domain transitions.

Assumptions. We assume that an analyst prepares an offline reference database D , where each entry associates a CVE with the corresponding vulnerable function identified from the security patch’s diff. This is because CVE reports often mention only the crash site instead of the actual root-cause function. For each entry in D , the reference functions are compiled using the same compilers under optimization levels $-O0$ through $-O3$. **MJ:** We refer to these eight compiler/optimization configurations as the *default builds*. We retain as references the default-build instances in which the vulnerable function remains as a standalone routine. During this offline

construction, we also record the inlining types exhibited by each vulnerability across the default builds. To ensure that each entry corresponds to a single unambiguous vulnerable function, we ① restrict D to CVEs whose patches modify only vulnerable functions (at most two) and ② manually exclude any non-vulnerable functions appearing in the same patch. **MJ:** We use DWARF metadata during reference database construction to identify these inlining relationships and during evaluation to establish ground truth. DWARF metadata from the target binary is never accessed at query time.

Scope. Zero-day vulnerability discovery is a separate problem and falls outside the scope of this work. We do not explicitly address obfuscated binaries (e.g., code virtualization or adversarial instruction substitution), non-x86-64 architectures, or automated patch synthesis and deployment. We focus solely on localizing vulnerable code regions, identifying *where* the vulnerability resides within the binary, while generating and applying the fixes is left to existing binary-patching frameworks [28].

IV. MOTIVATING EXAMPLE

Figure 6a illustrates the instruction-level layout (as recorded in DWARF) of the `read_markers` function (i.e., the inliner) in the `djpeg` binary compiled with Clang `-O1`. In this executable, the vulnerable function `get_sos` (CVE-2012-2806; heap-based buffer overflow) has been inlined together with seven non-vulnerable callees (`get_dac`, `get_dht`, `get_dqt`, `get_dri`, `get_app0`, `get_app14`, and `skip_variable`). As a result, `get_sos` no longer exists as an independent routine, falling under a Type II as in Figure 1. In this example, the inlined body of `get_sos` occupies only 22.5% of the inliner. Moreover, the inlined body has been fragmented into two non-contiguous regions, which arises from branch-processing routines in the inliner, such as loop counters, error handling, and conditional jumps.

Limitations of Prior Approaches. Precisely localizing the vulnerable code originating from `get_sos` is challenging using the previous ML-based approaches due to several common properties of compiler inlining. ① *Vanished function boundary:* The only function emitted at this site is `read_markers`, whose name and overall semantics bear little resemblance to `get_sos`. Existing BCSD systems [5], [3], [12], [11] compare a standalone reference against functions recovered from the target binary; consequently, `read_markers` is either discarded or assigned a similarity score far below the matching threshold, causing the vulnerability to be reported as absent. This challenge is further exacerbated when the same vulnerable function body is replicated across *multiple* inliners (i.e., inlining at several distinct call sites). Indeed, we confirmed that none of the prior work explicitly attempts to identify disappeared-but-vulnerable inlined(s) (i.e., Type II). ② *Whole-function dilution:* Whole-function embeddings measure similarity between the standalone vulnerable function and a host function in which the remaining 77.5% of the code originates from unrelated co-inlines and caller-side logic. As additional inlines accumulate, under aggressive optimization levels, such as `-O2`

and `-O3`, the similarity score becomes increasingly diluted. ③ *Token-window blindness:* Recent learning-based comparators typically operate on a fixed-length token window containing only a few hundred tokens (e.g., 250 in BCSD pipelines, 512 in jTrans-style models), truncating the remainder of the function once the limit is exceeded. In this example, the inlined body of `get_sos` is located near the *end* of `read_markers`, highlighting that a fixed-window encoder captures only the leading co-inlines and never observes the vulnerable code.

V. PINPOINT DESIGN

Design Objectives. To tackle the challenges described in Section IV, we design PinPoint, an inlining-resilient vulnerability localization system, with the following requirements in mind: ① *Boundary independence:* the system must localize vulnerable code even when the vulnerable function no longer preserves its original function-level boundary of its own (Type II), rather than relying on a one-to-one correspondence with a surviving function; ② *Dilution resistance:* because an inlined vulnerable body may constitute only a small portion of its host function (e.g., 22.5% in Figure 6a), code similarity must operate on sub-function regions instead of the entire inliner; and ③ *Full-function coverage:* the vulnerable region may appear anywhere in the inliner, including near the end of the function; therefore, the system must examine the whole function rather than a fixed-length prefix alone. **MJ:** Type I preserves the vulnerable function as a standalone routine, so conventional function-level matching remains applicable. Types II–IV, in contrast, involve inlining that can either remove the vulnerable function boundary or alter its function body with inlined code. PinPoint therefore begins these inlining-related cases with sub-function search at Stage 2.

PinPoint Overview. Figure 2 outlines the overview of PinPoint, a three-phase pipeline that utilizes prior BCSD techniques when applicable, and otherwise searches for the entire vulnerable reference *within* a candidate function using progressively finer-grained strides. **MJ:** For vulnerabilities exhibiting any of Types II–IV across their default builds, PinPoint additionally constructs `-fno-inline` references under the same eight compiler/optimization configurations. The inlining types recorded in D determine the entry stage: vulnerabilities exhibiting only Type I across their default builds begin at Stage 1, whereas those exhibiting any of Types II–IV begin directly at Stage 2. Before matching at the selected entry stage, PinPoint applies the size-ratio pruning described below. The pipeline advances to the next stage only when the current phase fails to localize the vulnerability with a confidence score above the threshold τ . By design, PinPoint can be compatible with any BCSD model capable of producing fixed-dimensional function embeddings from a sequence of up to L (canonicalized) assembly tokens of f^V , computing a similarity metric (e.g., cosine similarity).

Size-ratio Pruning. **MJ:** The additional references and sliding-window searches increase the analysis cost. To reduce this cost, PinPoint uses function size to avoid unnecessary

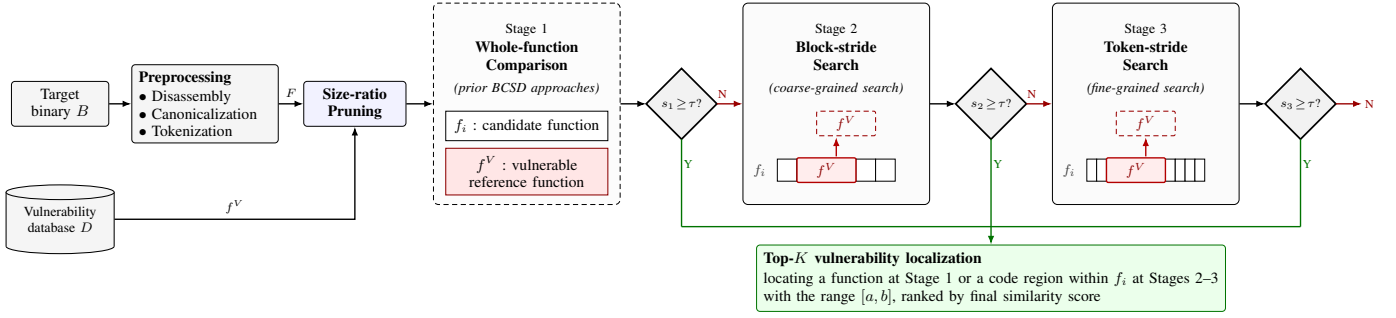


Fig. 2: Overview of PinPoint. Given a target binary B , PinPoint first applies disassembly, canonicalization, and tokenization to produce a set of candidate functions F , each of which is matched against vulnerable references drawn from the offline vulnerability database D . MJ: Before matching, PinPoint applies size-ratio pruning to skip a candidate function that is too small relative to a given vulnerable reference. MJ: For each vulnerable reference $f^V \in D$, the entry stage is determined by the inlining types observed for the corresponding vulnerability across its default builds. Vulnerabilities exhibiting only Type I begin at Stage 1, *Whole-function Comparison* (prior BCSD approaches), whereas those exhibiting any of Types II–IV begin directly at Stage 2, *Basic-block-stride Search* (coarse-grained search). MJ: From the selected entry stage, PinPoint advances to the next stage only when the current stage fails to satisfy $s_i \geq \tau$; Stage 3 performs *Token-stride Search* (fine-grained search). In Stages 2 and 3, an f^V -sized sliding window (solid red box) moves across f_i , with its next position (dashed box, offset by one stride) drawn directly above; the stride unit is one basic block in Stage 2 and one token in Stage 3. MJ: PinPoint reports the Top- K vulnerability localization results: a function located at Stage 1 or a code region within f_i located at Stages 2–3, with the range $[a, b]$, ranked by their final similarity scores.

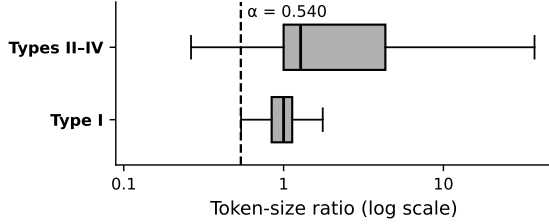


Fig. 3: MJ: Distribution of the token-size ratio $|T(f)|/|T(f^V)|$ for ground-truth candidate functions in Type I and inlining-related Types II–IV. A ratio below 1 indicates that the candidate function f is smaller than the vulnerable reference f^V . The dashed line marks the Type I Tukey lower fence ($\alpha = 0.540$) in log space. PinPoint uses this value as the size-ratio pruning threshold and skips matching f against f^V when $|T(f)| < \alpha|T(f^V)|$.

matching. Let $T(f)$ denote the canonicalized token sequence of a candidate function f , and $|T(f)|$ its number of tokens. Given a vulnerable reference f^V , PinPoint skips matching f against f^V when

$$|T(f)| < \alpha|T(f^V)|,$$

where α denotes the size-ratio threshold. MJ: We derive α from Type I, where the vulnerable function remains standalone and therefore provides a baseline without the size increase caused by inlining. As shown in Figure 3, the Type-I distribution has a Tukey lower fence of 0.540 in log space. Values below this threshold correspond to lower-tail outliers in the Type-I distribution. In contrast, Types II–IV generally exhibit larger size ratios because the target function may contain

additional code introduced through inlining. We therefore use $\alpha = 0.540$ for all types and skip matching a candidate function whose token size is below 54% of the vulnerable reference. This pruning may skip a valid match; Section VII-D evaluates its effect on retrieval effectiveness and runtime.

(Stage 1) Whole-function Comparison. PinPoint embeds the candidate function f_i and the vulnerable reference f^V , and computes their similarity score, yielding $s_i(f, f^V)$. The entirety of each f is disassembled, canonicalized (if required), and tokenized in preparation for subsequent matching. Note that f^V may be truncated to a sequence of up to L tokens while preserving basic-block boundaries, where L corresponds to the input limit imposed by conventional token-based BCSD models for function-pair comparison. This stage mirrors the matching procedure in such approaches, thereby naturally handling Type I cases in which f^V retains its original function boundary. Accordingly, PinPoint directly reuses prior BCSD techniques without introducing additional overhead. If any reference produces a similarity score satisfying $s_1 \geq \tau$, PinPoint terminates and reports the match. Otherwise, the vulnerable body is likely diluted or truncated within a larger inliner, and PinPoint proceeds to the basic-block-stride search.

(Stage 2) Basic-block-stride Search. MJ: Stage 2 handles Type I cases unresolved by Stage 1 and vulnerabilities that exhibit any of Types II–IV across their default builds. PinPoint searches for f^V within f by sliding a window of k basic blocks across the function, where k denotes the number of basic blocks in f^V , using a stride of one basic block (Figure 2). Each window is compared against f^V , and PinPoint records the maximum similarity score $s_2(f_i, f^V)$ together with the matched basic-block range. If f contains fewer than k basic

blocks, the entire function is treated as a single window. By construction, each basic-block window is boundary-aligned, meaning that it begins and ends at valid basic-block boundaries within f rather than at arbitrary instruction offsets. When the inlined body of f^V occupies a contiguous sequence of basic blocks, a corresponding window aligns with the vulnerable region, yielding $s_2 \geq \tau$ and thereby localizing the vulnerability to an address range within the inliner. This coarse-grained pass evaluates only one window per basic block. If a vulnerable code were not localized during Stage 2, PinPoint proceeds to the next stage.

(Stage 3) Token-stride Search. Stage 3 performs a token-level sliding-window search by moving a window of size $\min(|f^V|, L)$ across f with a stride of one token. This fine-grained search enables the window to align with the start of *any* token range, regardless of the basic-block layout selected by the compiler, thereby attempting to recover inlinees whose bodies are fragmented into non-contiguous regions (Section IV). For each candidate window, PinPoint computes the similarity score and records the maximum value, $s_3(f_i, f^V)$, together with the corresponding token range, which is subsequently mapped back to an address range of $[a, b]$ within f . This stage constitutes the finest-grained and most computationally expensive phase of the pipeline. Notably, a function containing n tokens can produce $n - \min(|f^V|, L) + 1$ candidate windows, substantially more than the number evaluated during the basic-block-stride search. To reduce online analysis overhead, PinPoint precomputes the stride-one embeddings of every $f^V \in D$ offline, lowering the per-query cost to a single forward pass over the sliding windows of f .

Decisions. PinPoint terminates the pipeline whenever a similarity score reaches the threshold τ . Accordingly, the final score is defined as $S(f, f^V) = s_i$ for the last executed stage i , and the final matched region corresponds to the region identified at that stage: the entire function f in Stage 1, a basic-block range in Stage 2, or a token range in Stage 3. The final decision is *evaluated from two perspectives* (Figure 2): ① *Top-k Ranking:* For each query function f , PinPoint ranks candidate functions in the target binary according to their similarity scores ($s_i \geq \tau$), and returns Top-k candidates. A query is considered correctly localized at Top- K if its ground-truth target appears among the top- K results. ② *Vulnerable Code Range Localization:* At Stage 1, PinPoint reports the entire function boundary as the vulnerable code range (*i.e.*, Type I). At Stage 2 or Stage 3, however, PinPoint considers localization successful only if the reported address range $[a, b]$ (*i.e.*, the sliding window) overlaps the inlined function within the host function f . In practice, multiple code regions may attain the same maximum similarity score against a reference function. To avoid such ties, we select the sliding window that contains *the largest number of occurrences of the maximum score* (Figure 4).

Hyperparameters. The PinPoint pipeline exposes several hyperparameters, including the token-window limit L (determined by the underlying BCSD model), the size-ratio

threshold α , the similarity threshold τ , the ranking depth K , and the basic-block window size k , defined as the number of basic blocks in f^V . The sliding strides are fixed at one basic block and one token for Stage 2 and Stage 3, respectively.

VI. IMPLEMENTATION

PinPoint System. We implement PinPoint in Python as a backbone-agnostic localization layer built atop three pretrained token-based BCSD models: BinShot [4], SAFE [3], and Trex [5]. We use their publicly released weights without fine-tuning. Stage 1 directly applies each backbone’s native whole-function embedding (Type I), whereas the core contribution of PinPoint lies in the sliding-window searches of Stages 2 and 3 (Types II–IV). For each candidate function, PinPoint constructs basic-block-stride and token-stride windows over the *entire* instruction sequence, re-embeds each window using the same backbone, computes similarity against the whole vulnerable reference, and advances to the next stage only when the current stage fails to satisfy $\tau = 0.99$ (Section VIII). *Additionally, we set the size ratio as $\alpha = 0.54$ to improve performance by filtering out small-sized functions.* To provide a uniform representation across heterogeneous preprocessing pipelines, we use Ghidra 11.4’s headless analyzer [29] for BinShot and Trex to recover per-function assembly, function and basic-block boundaries, and the basic-block-to-instruction mappings required by Stages 2 and 3. We further apply BinShot’s operand-normalization rules to the Ghidra-extracted assembly, whereas SAFE retains its original radare2 v4.2.1 (r2pipe v1.6.2) [30] preprocessing pipeline. The token budget L follows each backbone’s input limit: $L = 253, 150$, and 512 for BinShot, SAFE, and Trex, respectively, and constrains only the vulnerable reference and sliding-window size, never the candidate function itself. To reduce the overhead of token-stride search, PinPoint precomputes stride-one embeddings for every $f^V \in D$ offline. Note that we construct ground truth offline using `pyelftools` [31] over DWARF metadata (See Section A in Appendix).

Baseline Configuration. The graph-based approaches, VulHawk [12] and HermesSim [11], are evaluated using their original pipelines and are not integrated into PinPoint’s sliding-window stages. All baselines use publicly released weights without fine-tuning and retain the preprocessing pipelines specified by their authors, so performance differences primarily reflect the matching strategy.

VII. EVALUATION

We evaluate PinPoint on a 64-bit Ubuntu 26.04 LTS system equipped with dual Intel Xeon 6530P 32-Core processors, 512GB RAM, and a single NVIDIA H200 NVL GPU with 141GB of graphics memory.

A. Experimental Setup

Binary Corpus and Vulnerability Database. We construct the corpus from VulnFix [32] and Vul4C [33], covering 73 CVEs across nine popular open-source projects and 22

TABLE I: Summary of vulnerable functions involved in inlining-relationships (Types II-IV). As defined in Figure 1, $\rightarrow/\leadsto$ denote direct/nested inlining. The rightmost column indicates compiler and optimization level (c: clang, g: GCC).

Project:Binary	Version	CVE (CWE)	Type	Inliner	Nested	Inlinee	C[O]
binutils:nm-new	2.29.51	2017-14939 (125)	III	decode_line_info	→	line_info_add_file_name	c[1-3], g2
		2017-14940 (476)	III	_bfd_elf_slurp_version_tables	→	_bfd_elf_swap_verdaux_in	c[1-3], g[2-3]
		2017-15020 (125)	II	dwarf1_unit_find_nearest_line	→	parse_line_table	c[1-3], g[1-3]
		2017-15021 (125)	II	bfd_get_debug_link_info	→	bfd_get_debug_link_info_1	g3
		2017-15022 (476)	II	_bfd_dwarf2_find_nearest_line	→	parse_comp_unit	c[1-3], g[1-3]
		2017-15022 (476)	III	scan_unit_for_symbols	→	arange_add	c[1-3]
		2017-15023 (476)	II	decode_line_info	→	read_formatted_entries	g3
		2017-15023 (476)	III	read_formatted_entries	→	read_l_byte	c[1-3]
		2017-15024 (835)	III	scan_unit_for_symbols	→	arange_add	c[1-3]
		2017-15025 (369)	III	decode_line_info	→	line_info_add_file_name	c[1-3], g2
2017-15938 (119)	II	_bfd_dwarf2_find_nearest_line	→	parse_comp_unit	c[1-3], g[1-3]		
binutils:readelf	2.28.51	2017-6965 (119)	IV	apply_relocations	→	target_specific_reloc_handling	c[1-3], g[1-3]
		2017-9043 (20)	II	process_notes_at	↗	print_gnu_build_attribute_name	c[2-3], g[1-3]
		2017-9043 (20)	II	process_note	→	print_gnu_build_attribute_name	c1
	2.30.51	2018-10372 (125)	III	process_cu_tu_index	→	add_shndx_to_cu_tu_entry	c[1-3], g[2-3]
2.32.51	2019-9077 (787)	II	process_object	↗	process_mips_specific	g[1,3]	
2019-9077 (787)	III	process_mips_specific	→	find_section_by_type	c3, g2		
coreutils:pr	8.25	gnubug-25023 (N/A)	II	print_files	→	init_parameters	c[1-3], g[1-2]
coreutils:shred	8.27	gnubug-26545 (N/A)	II	do_wipefd	↗	fillpattern	c[1-3]
		gnubug-26545 (N/A)	II	dopass	→	fillpattern	g[1-3]
coreutils:split	8.25	gnubug-25003 (N/A)	II	main	→	bytes_chunk_extract	c[1-3], g[1-3]
jasper:imginfo	1.900.5	2016-10248 (476)	III	jpc_tsfb_synthesize	→	jpc_tsfb_synthesize2	g3
	1.900.17	2016-10251 (190)	II	jpc_pi_next	→	jpc_pi_nextcprl	c[1-3], g[1-3]
	1.900.13	2016-8691 (369)	III	jpc_siz_getparms	→	jpc_getuint8	g3
	1.900.3	2016-8692 (369)	III	jpc_siz_getparms	→	jpc_getuint8	g3
	1.900.5	2016-8887 (476)	III	jp2_box_get	→	jp2_box_destroy	c[1-3], g[2-3]
	1.900.13	2016-9390 (20)	III	jpc_siz_getparms	→	jpc_getuint8	g3
		2016-9391 (N/A)	IV	jpc_bitstream_getbits	→	jpc_bitstream_getbits	g3
		2016-9391 (N/A)	II	jpc_bitstream_inalign	→	jpc_bitstream_getbits	c[1-3], g[2-3]
	1.900.15	2016-9394 (20)	III	jpc_siz_getparms	→	jpc_getuint8	g3
	1.900.13	2016-9396 (N/A)	III	jpc_cox_getcompparms	→	jpc_getuint8	c[1-3], g[2-3]
1.900.15	2016-9398 (617)	II	jpc_dec_decodepkts	→	jpc_dec_decodepkt	c[1-3], g[1-3]	
1.900.22	2016-9560 (787)	II	jpc_dec_process_sod	→	jpc_dec_tileinit	c[1-3]	
libarchive:bsdtar	3.2.2	2016-10349/10350 (119)	II	archive_read_format_cab_read_header	→	cab_read_header	c[1-3], g[1-3]
	3.2.0	2016-5844 (190)	II	archive_read_format_iso9660_read_header	→	choose_volume	c[1-3], g[1-3]
libjpeg:djpeg	1.2.1	2012-2806 (787)	II	read_markers	→	get_sos	c[1-3], g[1-3]
libming:lstmp3	0.4.8	2016-9264 (119)	III	printMP3Headers	→	skipBytes	c[1-3], g[2-3]
	0.4.7	2016-9265 (369)	III	printMP3Headers	→	skipBytes	c[1-3]
	0.4.7	2016-9266 (189)	III	printMP3Headers	→	skipBytes	c[1-3]
libming:listsdf	0.4.7	2016-9828 (476)	IV	outputBlock	→	outputBlock	g3
		2016-9828 (476)	III	outputBlock	→	outputSWF_UNKNOWNBLOCK	c[1-3]
		2016-9831 (119)	II	readRect;	→	readBits	c3
		2016-9831 (119)	II	readSBits	→	readUInt8	c[1-3]
				_dumpBytes; readBytes; readDouble;			
		readEncSInt32; readEncUInt30; readEncUInt32;					
		readFloat; readSInt16; readSInt32;					
		readSInt8; readSizedString; readString;					
		readUInt16; readUInt32; silentSkipBytes					
libtiff:gif2tiff	4.0.1	2013-4243 (119)	III	readgifimage	→	initcolors	c[1-3], g3
libtiff:tiff2pdf	4.0.7	2016-10094 (189)	III	t2p_readwrite_pdf_image_tile	→	t2pWriteFile	c[1-3], g[2-3]
libtiff:tiffcp	4.0.7	2016-10093 (119, 190)	IV	readContigTilesIntoBuffer	→	cpStripToTile	c[1-3], g[2-3]
		2016-10093 (119, 190)	II	writeBufferToContigTiles	→	cpStripToTile	c[1-3], g[2-3]
		2016-10270 (125)	II	TIFFReadDirectory	→	ChopUpSingleUncompressedStrip	g[1-3]
		2017-5225 (119, 122)	II	main	↗	pickCopyFunc	c[1-3], g[1-3]
		2017-7598 (369)	III	TIFFReadDirEntryCheckedRational;	→	TIFFReadDirEntryData	c[1-3], g3
		2017-7602 (190)	II	TIFFReadDirEntryCheckedSrrational	→	TIFFReadRawStrip1	g3
libtiff:tiffcrop	4.0.7	2016-10092/10271/10272 (119)	II	loadImage	→	readContigStripsIntoBuffer	c[1-3], g[1-2]
		2016-10092/10271/10272 (119)	II	main	↗	readContigStripsIntoBuffer	g3
		2016-5321 (119)	II	loadImage	→	readSeparateTilesIntoBuffer	c3, g2
2016-5321 (119)	II	main	↗	readSeparateTilesIntoBuffer	c[1-2]		
libtiff:tiffinfo	3.8.0	2006-2025 (N/A)	II	TIFFFetchByteArray	→	TIFFFetchData	g3
	2006-2025 (N/A)	II	TIFFReadDirectory	↗	TIFFFetchData	g3	
libtiff:tiffmedian	4.0.7	2016-10267 (369)	III	OJPEGPreDecode	→	OJPEGLibjpegSessionAbort	c[1-3], g[2-3]
	bugzilla-2611 (N/A)	III	OJPEGPreDecode	→	OJPEGLibjpegSessionAbort	c[1-3], g[2-3]	
libtiff:tiffsplit	4.0.7	2016-10095 (119)	III	TIFFReadDirectory	→	ChopUpSingleUncompressedStrip	g[2-3]
libxml2:xmlint	2.9.0	2012-5134 (119)	III	xmlParseAttValueComplex	↗	xmlFatalErrMsgInt	g3
		2012-5134 (119)	II	xmlParseAttValueInternal	→	xmlParseAttValueComplex	c[1-3], g[1-2]
	2.9.4	2016-1838 (125)	III	xmlParseEndTag2	→	nsPop	c[1-3]
		2016-1839 (125)	IV	htmlParseName	→	htmlParseNameComplex	c[1-3], g[1-3]
		2017-5969 (476)	III	xmlDumpElementContent	→	xmlErrValid	c[1-3], g[2-3]
zziplib:unzipcat-mem	0.13.62	2017-5974 (119)	II	zzip_mem_disk_load	↗	zzip_mem_entry_extra_block	c[1-3], g[2-3]
		2017-5975 (787)	II	zzip_disk_entry_strdup_name;	→	zzip_disk_entry_to_file_header	c[1-3], g[2-3]
		2017-5975 (787)	II	zzip_disk_entry_to_data;	↗	zzip_disk_entry_to_file_header	c1, g3
				zzip_disk_entry_fopen			
				zzip_disk_findfile;			
				zzip_disk_findmatch			
		2017-5975 (787)	II	zzip_disk_fopen	↗	zzip_disk_entry_to_file_header	c3, g3
2017-5975 (787)	II	zzip_mem_disk_load	→	zzip_mem_entry_new	c[1-3], g[1-3]		
2017-5976 (787)	II	zzip_mem_disk_load	↗	zzip_mem_entry_extra_block	c[1-3], g[2-3]		

TABLE II: Summary of vulnerability-related target functions by type. *#Instances* counts target-function instances over six target builds per configuration (two compilers $\times$ -O1/-O2/-O3; -O0 excluded). *#Functions* counts distinct recovered target-function names. The Total row deduplicates entries across types. Types I-IV are defined in Section III.

Type	#Instances	#Projects	#Functions	#CVEs
I	291	8	59	57
II	195	9	48	32
III	91	5	21	28
IV	19	5	5	5
Total	596	9	108	73

release lines: binutils, coreutils, jasper, libarchive, libjpeg-turbo, libming, libtiff, libxml2, and zziplib. Table I summarizes the vulnerable functions involved in inlining relationships, including their project, binary, version, CVE, inlining type, and compiler/optimization configurations. For the binary corpus, we compile binaries using GCC and Clang under -O1, -O2, and -O3, yielding six versions per each. Using Ghidra [29], we recover 224,188 binary functions, including 223,592 non-vulnerable functions, of which 38,516 (17.2%) carry inlining relationships. Among the remaining 596 vulnerability-relevant functions, Table II summarizes detailed statistics of each inlining type, including the number of instances, projects, functions, and CVEs. For the reference database D , we manually inspect every security patch diff to identify 74 root-cause (*i.e.*, vulnerability-triggering) functions across 12 CWEs (Table VI), discarding vulnerability-irrelevant functions. We compile each in eight versions (two compilers, -O0-3); unlike the corpus, D retains -O0 so a target may match a reference from any optimization level. For vulnerabilities exhibiting any of Types II-IV across their default builds, we additionally compile binaries with -fno-inline under the same configurations and add the resulting vulnerable functions to D as optimized, boundary-preserving references of the vulnerable functions that more closely match the optimized targets (Section V).

Baselines. We instantiate PinPoint with three token-based BCSD approaches, namely SAFE [3], Trex [5], and BinShot [4] (denoted PinPoint-SAFE, PinPoint-Trex, and PinPoint-BinShot), and compare against two standalone graph-based baselines, VulHawk [12] and HermesSim [11]. In principle, any BCSD model can serve as the whole-function comparison in PinPoint’s Stage 1; however, the primary advantage of PinPoint emerges in Stages 2 and 3, which slide a reference-sized window across the candidate function and *re-embed* each window independently. Token-sequence backbones naturally support this operation, whereas VulHawk (*e.g.*, Hex-Ray’s microcode) and HermesSim (*e.g.*, Ghidra’s P-code) encode entire function graphs, rendering sliding sub-function windows impractical without per-window re-lifting and handling out-of-distribution partial graphs (Section VIII). Consequently, a graph-based model restricted to Stage 1 effectively reduces to its standalone whole-function form, which we report directly.

All models are evaluated using the authors’ released pretrained weights on the same evaluation corpus.

Evaluation Metrics. We evaluate PinPoint and the baseline models from two metrics. ① *Top-K Retrieval Accuracy* ($K \in \{1, 5, 10\}$) and *MRR*: Following the decision process in Section V, PinPoint ranks the Top-k candidate functions in a target binary by similarity. For similarity-based matching, we use a fixed decision threshold of $\tau = 0.99$. We additionally report mean reciprocal rank (MRR), computed as the reciprocal rank of the ground-truth function across the evaluated target instances.

MJ: ② *Vulnerability Range Localization*: we evaluate PinPoint-BinShot queries for which the ground-truth target function is ranked Top-1. For each query, we evaluate the reported range or ranges produced by the final executed stage. A Stage 1 result covers the entire target function, whereas Stages 2 and 3 select the reported range or ranges from the windows evaluated at the final sliding-window stage. MJ: At Stages 2 and 3, multiple windows can attain the same maximum similarity score. Because the similarity score alone cannot distinguish among these windows, we use their relative positions to select the reported range or ranges. Let $\mathcal{W}_{\max}$ denote the set of windows attaining the maximum similarity score at that stage. For each $w \in \mathcal{W}_{\max}$, let $s(w)$ and $e(w)$ denote the indices of the first and last elements covered by w , respectively, where the elements are basic blocks at Stage 2 and tokens at Stage 3. We define the *containment count* of w as

$$c(w) = |\{w' \in \mathcal{W}_{\max} \mid s(w) \leq s(w') \leq e(w)\}|. \quad (1)$$

MJ: Thus, $c(w)$ counts the maximum-score windows whose start indices fall within w , including w itself. We retain the window or windows with the largest containment count:

$$\mathcal{W}^* = \arg \max_{w \in \mathcal{W}_{\max}} c(w). \quad (2)$$

MJ: If multiple windows share the largest containment count, all are retained. The selected windows are mapped to their corresponding byte ranges in the target function, which constitute the reported ranges. This selection uses only similarity scores and window indices, without ground-truth information. MJ: A query is successfully localized if at least one reported range overlaps the DWARF-derived vulnerable bytes. We report localization accuracy as the fraction of the evaluated Top-1 queries that satisfy this criterion. Complete coverage of the DWARF-derived vulnerable bytes is not required. MJ: Among successfully localized queries, we further examine whether higher window similarity scores are associated with vulnerable code. For these queries, we collect all Stage 2 and 3 windows actually evaluated during the search and record whether each window’s byte range overlaps the DWARF-derived vulnerable bytes. We compute the point-biserial correlation between the window similarity score and this binary overlap indicator, and report the result overall and separately for Types II and IV.

Research Questions. We evaluate PinPoint with the following three research questions and further illustrate qualitative matching behavior in Section VII-E.

- **RQ1 (Effectiveness):** How accurately does PinPoint MJ: retrieve the vulnerable code compared with existing BCSD baselines, MJ: as measured by Top- K accuracy and MRR, and how does performance vary across the four inlining types (Section VII-B)?
- **RQ2 (Vulnerability Range Localization):** How accurately does PinPoint localize vulnerable code within the target function (Section VII-C)?
- **RQ3 (Efficiency):** What runtime overhead does PinPoint incur per comparison, and how does MJ: size-ratio pruning affect this cost and MJ: retrieval effectiveness (Section VII-D)?

B. Effectiveness

MJ: Top- K Retrieval Accuracy. MJ: Table III reports the overall and type-wise Top- K retrieval accuracy and MRR of PinPoint and the baseline models. MJ: At the aggregate level, PinPoint-BinShot improves Top-1 accuracy from 58.6% to 72.9%, whereas PinPoint-SAFE shows little change, from 46.1% to 45.6%. PinPoint-Trex also improves Top-1 accuracy from 43.6% to 48.4%. Thus, the aggregate effect of PinPoint varies across the underlying BCSD backbones. MJ: The type-wise results, however, reveal a common pattern. Except for HermesSim, Type II has the lowest Top-1 accuracy among the reported model configurations. Type II also accounts for 195 of the 596 target-function instances (32.7%). As discussed in Section III, Type II embeds the vulnerable function within a non-vulnerable function, where it no longer survives at a recoverable function boundary. We therefore examine Type II separately below.

MJ: Retrieval of Embedded Vulnerable Inlinees. MJ: Type II directly evaluates retrieval when the vulnerable function no longer survives at a recoverable function boundary. With BinShot, PinPoint increases Type II Top-1 accuracy from 26.9% to 65.5%. The improvement also persists at larger retrieval depths, with more Type II target functions appearing within the Top-10 results. MJ: SAFE shows the same Type II pattern: Top-1 increases from 7.0% to 14.6%. In contrast, its Type I Top-1 decreases slightly from 65.7% to 62.7%. For Type I cases whose Stage 1 score remains below τ , PinPoint proceeds to the sliding-window stages, which can change the final ranking. This behavior is consistent with the slight Type I decrease observed with SAFE. MJ: Trex likewise improves Type II Top-1 accuracy from 9.9% to 16.8%. Across BinShot, SAFE, and Trex, PinPoint therefore consistently improves Type II retrieval over the corresponding BCSD backbone alone. MJ: The graph-based baselines show different behavior on Type II: VulHawk achieves 3.8% Top-1 accuracy, whereas HermesSim reaches 78.4%. Thus, Type II is not uniformly difficult across all BCSD models. MJ: Overall, these results show that PinPoint’s sub-function search improves retrieval over its underlying BCSD backbones when the vulnerable function no longer survives at a recoverable function boundary.

C. Vulnerability Range Localization

MJ: Localization of Embedded Vulnerable Inlinees. MJ: Table IV reports the within-function localization accuracy of PinPoint-BinShot for reference queries whose ground-truth target function is ranked Top-1. Unlike Table II, which counts target-function instances, this table counts Top-1 reference queries. MJ: Types I and III both reach 100% because their ground-truth vulnerable ranges span the target function itself. Once the correct function is retrieved, any reported range within that function overlaps the ground truth. We therefore focus on Types II and IV, where a vulnerable inlinee must be localized within the retrieved function. MJ: For Type II, 872 of 1,128 Top-1 reference queries (77.3%) successfully localize the vulnerable code within the retrieved non-vulnerable inliner. The remaining 256 queries retrieve the correct target function but fail to localize the vulnerable inlinee within it. Thus, retrieving the correct target function alone is insufficient for vulnerability localization. MJ: For Type IV, 261 of 275 Top-1 reference queries (94.9%) report a range overlapping the DWARF-derived vulnerable bytes. The remaining 14 queries retrieve the correct vulnerable inliner, but no reported range overlaps the embedded vulnerable inlinee. Thus, sub-function localization remains necessary even when the retrieved outer function is itself vulnerable.

MJ: Similarity Scores across Sliding Windows. MJ: We next examine whether window similarity scores distinguish windows that overlap the vulnerable code from those that do not. Across the 540,777 windows collected from successfully localized queries, the point-biserial correlation between the similarity score and whether a window overlaps the DWARF-derived vulnerable bytes is 0.1969. The correlation is 0.3484 for the 250,978 Type II windows and 0.1426 for the 49,401 Type IV windows. MJ: The correlations are positive, but the overall relationship is weak. Windows overlapping the vulnerable code tend to receive higher similarity scores, while windows outside the vulnerable code can also receive high scores. A high similarity score alone therefore does not determine which window should be reported. MJ: Figure 4 illustrates this pattern for `pickCopyFunc` in `main`. Maximum-score windows appear both within and outside the ground-truth vulnerable ranges, while the range reported by PinPoint overlaps the vulnerable code. MJ: When multiple windows attain the maximum similarity score, the score alone cannot distinguish among them. PinPoint therefore uses the containment count defined in Section VII-A and reports the maximum-score window or windows with the largest count.

D. Efficiency

Amortized Latency. Figure 5 profiles PinPoint-BinShot as a representative instantiation, comparing execution with and without size-ratio pruning over 4,004,510 candidate-reference pairs before pruning. A real analysis matches a vulnerable function only against the candidates of the binary under inspection; we aggregate over the whole corpus to characterize throughput. We amortize both configurations over the

TABLE III: Type-wise and overall Top- K function-retrieval accuracy and MRR of PinPoint and baseline BCSD models. For each target-function instance, we measure rank as the best (lowest) rank over all references derived from the same vulnerable function. T1, T5, and T10 denote Top-1, Top-5, and Top-10 accuracy. We evaluate BinShot, SAFE, and Trex standalone and as PinPoint backbones; † marks graph-based baselines evaluated only with whole-function matching (Stage 1). Overall aggregates Types I–IV, and shading marks the best value in each column.

Model	Type I				Type II				Type III				Type IV				Overall			
	T1	T5	T10	MRR	T1	T5	T10	MRR	T1	T5	T10	MRR	T1	T5	T10	MRR	T1	T5	T10	MRR
BinShot	70.0	78.6	82.0	0.743	26.9	41.3	49.9	0.342	75.2	85.2	87.8	0.805	56.2	67.1	76.3	0.623	58.6	69.1	74.1	0.640
PinPoint-BinShot	76.1	79.4	82.2	0.781	65.5	74.2	79.8	0.698	88.3	90.8	92.2	0.895	69.6	79.9	82.7	0.742	72.9	77.7	81.0	0.754
SAFE	65.7	81.9	87.7	0.733	7.0	15.2	19.8	0.124	70.6	84.2	88.2	0.770	26.5	49.1	58.0	0.369	46.1	59.2	64.2	0.526
PinPoint-SAFE	62.7	79.4	84.7	0.702	14.6	32.3	43.6	0.240	72.8	85.5	90.2	0.787	36.8	59.5	70.7	0.480	45.6	60.6	67.1	0.527
Trex	56.3	70.8	77.0	0.638	9.9	22.4	30.5	0.170	60.0	74.2	78.8	0.669	38.5	64.7	70.7	0.501	43.6	58.0	64.5	0.511
PinPoint-Trex	68.2	81.2	88.4	0.745	16.8	40.9	56.8	0.291	67.0	81.4	87.3	0.744	45.8	67.6	75.1	0.560	48.4	64.1	73.0	0.562
VulHawk †	39.9	52.1	60.1	0.464	3.8	7.9	13.8	0.070	31.5	43.5	49.7	0.374	12.2	16.3	19.8	0.148	25.9	35.0	41.5	0.309
HermesSim †	76.0	77.2	77.8	0.767	78.4	79.4	79.9	0.793	79.0	79.2	80.1	0.793	85.9	86.2	86.2	0.863	69.2	70.0	70.6	0.698
Average	64.4	75.1	80.0	0.697	27.9	39.2	46.8	0.341	68.1	78.0	81.8	0.730	46.4	61.3	67.4	0.536	51.3	61.7	67.0	0.566

TABLE IV: MJ: Within-function vulnerability range localization of PinPoint-BinShot for reference queries whose ground-truth target appears at Top-1. $\#Queries$ denotes the number of such reference queries. A query is counted as *Correct* when at least one reported range overlaps the vulnerable region; complete coverage of the region is not required. Accuracy is the fraction of correct queries within each type.

Type	#Queries	#Correct	Accuracy
I	1,568	1,568	100.0%
II	1,128	872	77.3%
III	1,518	1,518	100.0%
IV	275	261	94.9%
Total	4,489	4,219	94.0%

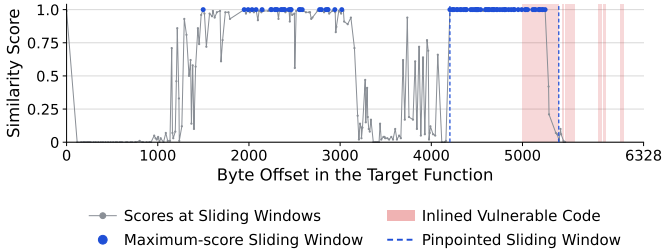


Fig. 4: Example of multiple code regions attaining the same maximum similarity score against a reference function during token-stride sliding window retrieval (Stage 2). In this case (CVE-2017-5225 [34]), the target function `main` is an inliner that inlines `pickCopyFunc`. PinPoint selects the sliding window (blue dotted line) because it contains the largest number of maximum-score occurrences, with 1.0 attained at 83 positions. The x-axis denotes byte offsets from the beginning of `main`, with each score plotted at the starting byte offset of its corresponding Stage 2 window. The scattered red regions indicate the inlines in which vulnerable code resides.

same pre-pruning pair count. This normalization is important because, without pruning, every pair enters the matching

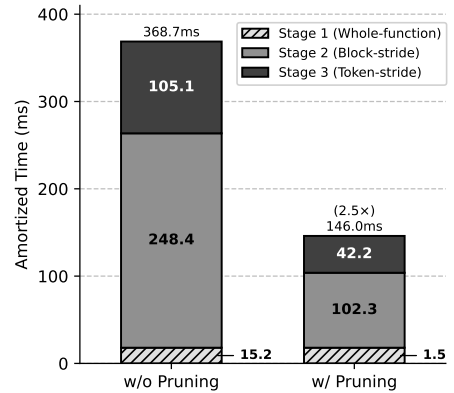


Fig. 5: Runtime comparison of PinPoint-BinShot with and without size-ratio pruning. Times are amortized over the same 4.0 million pre-pruning candidate-reference pairs. Speedup is relative to the configuration without pruning.

TABLE V: Aggregate Top- K retrieval accuracy and MRR of PinPoint-BinShot with and without size-ratio pruning.

Configuration	Top-1	Top-5	Top-10	MRR
w/o Pruning	72.7	77.5	81.1	0.754
w/ Pruning	72.9	77.7	81.0	0.754

cascade, whereas size-ratio pruning can discard a pair before any similarity score is computed. We report this end-to-end amortized latency rather than the cost of a single scorer call because the cascade can terminate at different stages once a candidate exceeds τ . Without pruning, the amortized runtime is 368.7 ms per pre-pruning pair; with size-ratio pruning, it drops to 146.0 ms, corresponding to a 2.5 $\times$ reduction in amortized latency.

Size-ratio Pruning Trade-off. Size-ratio pruning trades a small amount in valid-comparison coverage for a substantial reduction in scoring cost. Overall, our experiments show that

size-ratio pruning discards 73.9% of scoring operations and yields a $2.5\times$ speedup, while essentially preserving aggregate retrieval effectiveness. As in Table V, Top-1 accuracy changes only slightly, from 72.7% without pruning to 72.9% with pruning; Top-5 and Top-10 accuracy vary by at most 0.2 percentage; and MRR remains unchanged at 0.754. The slight increase in Top-1 accuracy occurs because pruning can also remove distractor candidates from the ranked list. However, the pruning trims comparisons involving ground-truth functions primarily for size-ratio outliers. Among those 8,839 comparisons, our pruning strategy skips 253 (2.9%) because their candidate/reference size ratios fall below the threshold. These skipped comparisons are concentrated in Type I: 220 of 4,341 Type I comparisons (5.1%) are pruned, whereas the skip rate remains below 1% for each of Types II–IV.

E. Case Studies

Effective Localization. Figure 6 presents four cases in which PinPoint successfully ranks the target vulnerable function at Top-1 and localizes the code region containing the vulnerable inlinee. Cases 1–3 and Case 4 correspond to Type II and Type IV, respectively. Recall that locating a vulnerable code region within a function always requires Stage 2 or Stage 3 of PinPoint. We showcase both basic-block-stride search (e.g., Case 2 and 3) and finer-grained token-stride search (e.g., Case 1 and 4).

(Case 1) Token-Stride Localization of a Fragmented Inlinee: Returning to the motivating example (Figure 6a), the inlinee contributes only 230 of the inliner’s 1,024 canonicalized tokens (22.5%), causing significant semantic dilution. Stage 2 aligns windows at basic-block boundaries, but an intervening `read_markers` block separates the two vulnerable regions, limiting the maximum similarity score to $0.77 < \tau$. Without block alignment constraints, Stage 3 identifies a token-level window spanning both fragmented regions as a contiguous token sequence, achieving a similarity score of 1.0 and ranking `read_markers` as Top-1. This example demonstrates the necessity of token-stride search when coarse-grained block-stride search is insufficient.

(Case 2) Basic-block-stride Localization of a Fragmented Inlinee: Under Clang -O2, `parse_comp_unit` is inlined into `_bfd_dwarf2_find_nearest_line` in `binutils’ nmnew` (CVE-2017-15022 [35]) and distributed across five non-contiguous ranges (Figure 6b). The vulnerable code accounts for 498 of the inliner’s 1,165 tokens (42.7%). Despite this fragmentation, Stage 2 achieves a maximum similarity score of 1.0, terminating the pipeline before Stage 3. The selected basic-block range overlaps two vulnerable regions and contains 204 of the 2,062 vulnerable bytes. Using `parse_comp_unit` as the reference, `_bfd_dwarf2_find_nearest_line` is ranked Top-1. Unlike Case 1, localization is resolved at the basic-block-stride level. This case demonstrates that coarse-grained search can identify vulnerable code even when the vulnerable inlinee is fragmented across multiple non-contiguous ranges.

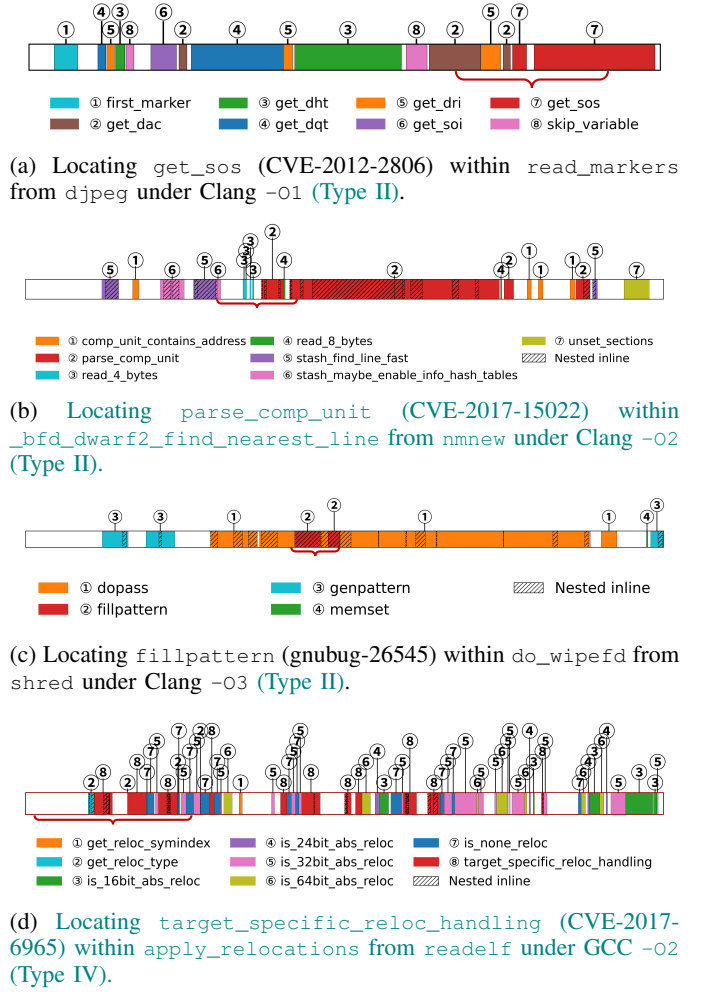


Fig. 6: Four representative cases of localizing vulnerabilities by PinPoint under function inlining. Cases (a)–(c) represent Type II, where a vulnerable inlinee is absorbed into a non-vulnerable function and no longer survives as an independent function, so PinPoint must locate it within the inliner. Case (d) belongs to Type IV, where the vulnerable inliner additionally contains a vulnerable inlinee. In every panel, each colored region is a contiguous range of an inlinee; distinct colors, hatched fill (in the parent’s color), and dotted boundaries denote top-level, nested, and deeper-nested inlinees, respectively. The (non-)continuous vulnerable inlinee is colored in red, and red braces indicate PinPoint’s reported ranges.

(Case 3) Token-Stride Localization under Nested Inlining: Under Clang -O3, `fillpattern` in GNU coreutils (gnubug-26545) is inlined into `do_wipefd` in `shred` through an intermediate function, `do_wipefd` → `dopass` → `fillpattern` such that neither callee survives as a standalone function (Figure 6c). Within the 1,003-token inliner, the nested `fillpattern` body contributes only 65 tokens (6.5%), making it difficult to identify by whole-function context alone. Stage 2 already places its highest-scoring basic-block range on the vulnerable region, but the maximum similarity score

is only 0.69, below the escalation threshold τ . The Stage 2 range overlaps 65.0% of the vulnerable code, indicating that coarse-grained search has found the right area but not with sufficient confidence or coverage. PinPoint therefore proceeds to Stage 3, where token-stride search refines the match within the same region. Stage 3 raises the maximum similarity score to 1.0 and the selected range overlaps 87.6% of the ground truth region. For the `fillpattern` reference, `do_wipefd` is ranked Top-1. This case shows that token-stride search can improve both confidence and vulnerable-byte coverage even when Stage 2 already identifies the correct neighborhood, and that PinPoint can localize a vulnerable inlined through two levels of inlining.

(Case 4) Basic-block-stride Localization in a Vulnerable Inliner: Under GCC `-O2`, `apply_relocations` in `binutils' readelf` (CVE-2017-6965 [36]) represents a Type IV case: the recovered outer function is itself vulnerable while also containing the vulnerable inlined `target_specific_reloc_handling` (Figure 6d). Thus, function-level retrieval can identify the vulnerable host, while finer-grained localization can further pinpoint the vulnerability-relevant region within that host. Using `apply_relocations` as the vulnerable reference, PinPoint ranks `apply_relocations` as Top-1. Stage 2 reaches a maximum similarity score of 1.0, causing the pipeline to terminate without executing Stage 3. The selected basic-block range spans blocks [36:102] of the 973-token target function and overlaps the vulnerable code. Unlike Case 3, the finer-grained token-stride search is unnecessary, as the vulnerable region is successfully localized at basic-block granularity. This case illustrates why PinPoint handles Type IV separately from Type II. Even when the vulnerable outer function remains recoverable, sub-function localization can identify the specific region associated with the inlined vulnerable code.

Failure Analysis. Our in-depth analysis reveals that 270 Top-1 failures in vulnerable-code localization fall into one of the following three cases: ① imprecise function boundary detection (17 cases), ② misranking of vulnerable windows by similarity score (91 cases), and ③ tie selection among maximum-score windows (162 cases).

(Case 1) Imprecise Function Boundary Detection: 17 failures occur before similarity scoring can examine the vulnerable region. In these cases, Ghidra recovers only a portion of the target function, restricting the sliding-window search to an incomplete function body. In our experiments, all 17 failures involve `bytes_chunk_extract` inlined into `main` of `coreutils-split`. For example, Ghidra recovers more than 93% of the original `main` body under `-O1`, but only 43-63% in several `-O2` and `-O3` builds. Under Clang `-O3` and GCC `-O2`, none of the vulnerable code is contained within the recovered basic blocks. Consequently, PinPoint cannot construct any Stage 2 or Stage 3 window that overlaps the vulnerable code. This failure mode originates from the binary front end rather than our approach, which propagates to any analysis pipeline that relies on accurate function boundaries; once the recovered

function omits the vulnerable bytes, no downstream window-scoring or tie-breaking strategy can recover them.

(Case 2) Misranking of Vulnerable Windows by Similarity Scores: 91 failures occur even though the vulnerable code is present in the recovered target function and is examined by the sliding-window search. In these cases, at least one evaluated window overlaps the vulnerable code, but every vulnerable-overlapping window receives a lower similarity score than the final maximum-score window. Consequently, the correct region is eliminated before the maximum-score tie-selection step is applied. Across these failures, the gap between the selected maximum score and the highest score assigned to any vulnerable-overlapping window is 0.138 on average, with a median of 0.03. These near misses indicate that the model often assigns nontrivial similarity to the vulnerable region, but still ranks an unrelated window higher. This failure mode therefore stems from window-level similarity scoring, rather than from incomplete function recovery or from the subsequent containment-count rule.

(Case 3) Tie Selection among Maximum-Score Windows: The remaining 162 failures occur after similarity scoring has already identified at least one plausible vulnerable location. In these cases, one or more windows overlapping the vulnerable code attain the maximum similarity score, but a different maximum-score window has a higher containment count and is selected instead. This is the largest failure category, accounting for 60.0% of the 270 localization failures. For example, `jpc_pi_nextcprl` in `jpc_pi_next` yields 97 maximum-score windows in one GCC `-O1` build. The selected window has a containment count of 28, whereas a maximum-score window overlapping the vulnerable code has a containment count of 17. Unlike Case 2, the similarity model does not rank all vulnerable windows below an unrelated region; instead, a vulnerable-overlapping window remains among the maximum-score candidates. The failure is therefore introduced by the deterministic containment-count selection rule, indicating that tie handling is the dominant remaining source of localization error once the vulnerable code is present and attains the maximum score.

VIII. DISCUSSION AND LIMITATIONS

Corpus Representativeness. We do not evaluate obfuscated binaries, alternative compiler families, non-x86-64 architectures, or other language ecosystems. Furthermore, binaries compiled with link time optimization, or optimization flags such as `-Os` and `-Ofast` may lack closely matching references in our dataset. Consequently, the observed distribution of inlining types and the PinPoint capability of localization may be corpus-dependent across different projects, compilers, and build configurations.

False Positives in Large Candidate Functions. Stages 2 and 3 score a candidate using the maximum similarity across all sliding windows. However, larger functions can introduce a multiple-comparisons effect; namely, unrelated regions gain more opportunities to produce spuriously high scores simply

because more windows are evaluated. As a result, a fixed threshold τ does not control false positives uniformly across functions of different sizes. Calibrating the score by the window count or candidate size is future work.

Sliding-Window Compatibility. PinPoint is instantiated on token-based BCSD backbones because Stages 2 and 3 slice and score windows over the model’s assembly-token sequence: Stage 2 needs recovered basic-block boundaries and a block-to-token mapping, and Stage 3 forms token-level windows directly. The released graph-based systems do not meet this requirement. VulHawk [12] encodes Hex-Rays microcode and HermesSim [11] P-code graphs, neither sliceable by the Stage 2 and 3 procedures, so we run them as standalone baselines. We leave extending sliding-window search to graph representations (e.g., subgraph matching) as our future work.

Dependence on Recovered Boundaries. PinPoint inherits the disassembler’s function and basic-block recovery: when Ghidra splits one source-level function across several (e.g., an interior region carved into a separate function), Stages 2 and 3 see only a fragment and recall is capped regardless of the matcher (Section VII-E). Any boundary-dependent pipeline inherently shares this limitation, regardless of our design.

Empirical Hyperparameter Choices. PinPoint exposes two parameters determined by the design: the escalation threshold τ and the size-ratio threshold α . The performance of PinPoint may vary depending on those parameters, as τ and α determine whether the search continues at a finer granularity rather than relaxing the match criterion. The remaining parameters are fixed by design: the token budget L follows the chosen backbone: the sliding strides are either a basic block size in Stage 2 or a token size in Stage 3.

Multiple Tied Windows. When multiple maximum-score windows share the largest containment count, PinPoint retains all of them as reported ranges. In our evaluation, such a query is considered successfully localized if at least one of the reported ranges overlaps the vulnerable code. This strategy avoids arbitrary tie-breaking, although it may leave multiple candidate ranges for a single query rather than a unique vulnerable region. Further improving the disambiguation among tied windows is a potential direction for future work.

IX. RELATED WORK

Binary Code Similarity Detection. A binary code similarity detection (BCSD) task determines whether two code snippets are semantically close across different compilers, optimization levels, or architectures. BCSD has been extensively studied [37] due to its broad range of security applications, including malware detection [10], code clone detection [38], vulnerability analysis [39], [12], or patch analysis [19]. Existing approaches learn function representations from instructions [40], traces [41], control or data flow graphs [8], [42], decompiled structures [43], [9], or LLM-derived embeddings [10], [44]. SAFE [3], BinShot [4], Trex [5], and jTrans [6] encode instruction sequences or execution-related signals using neural models, whereas HermesSim [11] and VulHawk [12] employ

graph- or IR-based representations to enhance semantic robustness. Asteria [9] instead models decompiled ASTs with a Tree-LSTM Siamese network, and KEENHash [10] compresses LLM-derived function embeddings into compact program-level representations for scalable analysis. These approaches provide effective function-level or program-level similarity matching for a variety of downstream security tasks. Guo et al. [45] conduct a large-scale empirical study of BCSD-based vulnerability detection in real-world firmware and examine the effects of vulnerable-function versions, search spaces, function sizes, and compilation toolchains. Likewise, our work leverages prior BCSD techniques [3], [5], [11], [12] as the baseline mechanism to detect the location of vulnerable code.

Vulnerability Localization. A vulnerability localization task aims to identify vulnerable (or buggy) code regions within a target executable, using reference information derived from vulnerable code, security patches, or source-to-binary mappings. VIVA [13] derives partial vulnerability and patch signatures from vulnerable and patched binaries through program slicing and trace refinement, while VulMatch [14] leverages source-level diffs and source-to-binary mappings to identify vulnerability-related binary instructions and construct fine-grained signatures. VPChecker [15] detects CVEs in shared libraries using public patch information, and further combines the results with binary-level function call graphs for reachability-based triage. Although these approaches localize evidence at the instruction, signature, or function level, they do not address the boundary mismatch introduced by compiler inlining, where vulnerable callee code may be embedded into a larger non-vulnerable caller. In contrast, our work localizes vulnerable code ranges even in the presence of inlining.

Patch Presence Detection. A patch presence detection task determines whether a known security patch has been applied to a target binary, rather than localizing the vulnerable region itself. FIBER [16] introduces the notion of patch presence testing by generating fine-grained signatures from localized patch-induced changes. Subsequent approaches have improved patch testing using symbolic execution or emulation, data-flow strands, patch-code localization, and semantic summaries. For example, Osprey [23] extracts cross-basic-block data-flow strands in VEX-IR, BinXray [17] and PatchDiscovery [24] compare target functions against patch-related traces or representative changed basic blocks. Meanwhile, PDiff [18], Robin [19], and PS³ [20] instead adopt semantic summaries or symbolic behaviors derived from pre- and post-patch binaries. SPAIN [22] summarizes vulnerability and patch patterns, whereas PLocator [21] localizes patch code using stable anchor values and CFG-context constraints. Unlike our work, these approaches primarily focus on determining patch status.

Function Inlining and Binary Analysis. Function inlining is a compiler optimization that alters function boundaries, thereby weakening structural assumptions commonly relied upon in binary analysis. Prior work has investigated the effects of inlining from multiple perspectives. Bingo [46] and Asm2Vec [7] expand selected callees to capture more complete

function semantics. Jia et al. [47] analyze the 1-to- n and n -to- n mappings introduced by function inlining and quantify their impact on binary similarity tasks. Abusabha et al. [25] further demonstrate that extremely aggressive inlining can be exploited to evade or degrade ML-based binary-analysis models across multiple security tasks. CI-Detector [48] models cross-inlining similarity with pattern-specific GNN estimators for leaf-, root-, and internal-inlining cases. These studies predominantly emphasize either the direct and indirect impact of inlining (e.g., robustness degradation) or the incorporation of inlining awareness to improve model performance. On the other hand, our work aims to seek all potential regions in which a vulnerable code may reside even under aggressive function inlining.

X. CONCLUSION

In this work, we study vulnerability localization under compiler function inlining, where a vulnerable function loses its boundary and survives only as inlined code within another function. We introduce a four-type taxonomy of post-inlining vulnerabilities and PinPoint, a backbone-agnostic vulnerability localization system. **MJ: PinPoint employs a size-ratio pruning with a threshold-gated three-stage cascade: it reuses whole-function BCSD for Type I, whereas inlining-related cases (Types II–IV) begin with block-stride search and proceed to token-stride search when needed to recover inlined, fragmented, and nested vulnerable code as a byte range within the host function.** Across 73 CVEs spanning nine projects, we demonstrate that PinPoint effectively and efficiently localizes vulnerable code even when compiler optimizations obscure function boundaries, without regressing on boundary-preserving cases.

REFERENCES

- [1] Zero Day Clock, “Zero Day Clock: Tracking time-to-exploit across CVEs,” 2024. [Online]. Available: <https://zerodayclock.com/>
- [2] The Mitre Corporation, “Common vulnerabilities and exposures (CVE),” 2024. [Online]. Available: <https://cve.mitre.org/>
- [3] L. Massarelli, G. A. Di Luna, F. Petroni, L. Querzoni, and R. Baldoni, “SAFE: Self-attentive function embeddings for binary similarity,” in *Proceedings of the 16th International Conference on Detection of Intrusions and Malware, and Vulnerability Assessment (DIMVA)*, 2019.
- [4] S. Ahn, S. Ahn, H. Koo, and Y. Paek, “Practical binary code similarity detection with BERT-based transferable similarity learning,” in *Proceedings of the 38th Annual Computer Security Applications Conference (ACSAC)*, 2022.
- [5] K. Pei, Z. Xuan, J. Yang, S. Jana, and B. Ray, “Learning approximate execution semantics from traces for binary function similarity,” *IEEE Transactions on Software Engineering (TSE)*, 2023.
- [6] H. Wang, W. Qu, G. Katz, W. Zhu, Z. Gao, H. Qiu, J. Zhuge, and C. Zhang, “jTrans: Jump-aware transformer for binary code similarity detection,” in *Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, 2022.
- [7] S. H. Ding, B. C. Fung, and P. Charland, “Asm2vec: Boosting static representation robustness for binary clone search against code obfuscation and compiler optimization,” in *Proceedings of the 40th IEEE Symposium on Security and Privacy (SP)*, 2019.
- [8] X. Xu, C. Liu, Q. Feng, H. Yin, L. Song, and D. Song, “Neural network-based graph embedding for cross-platform binary code similarity detection,” in *Proceedings of the 24th ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2017.
- [9] S. Yang, L. Cheng, Y. Zeng, Z. Lang, H. Zhu, and Z. Shi, “Asteria: Deep learning-based AST-encoding for cross-platform binary code similarity detection,” in *Proceedings of the 51st IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*, 2021.
- [10] Z. Liu, Q. Tang, S. Nie, S. Wu, L. F. Zhang, and Y. Tang, “KEENHash: Hashing programs into function-aware embeddings for large-scale binary code similarity analysis,” in *Proceedings of the 34th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, 2025.
- [11] H. He, X. Lin, Z. Weng, R. Zhao, S. Gan, L. Chen, Y. Ji, J. Wang, and Z. Xue, “Code is not natural language: Unlock the power of Semantics-Oriented graph representation for binary code similarity detection,” in *Proceedings of the 33rd USENIX Security Symposium (Security)*, 2024.
- [12] Z. Luo, P. Wang, B. Wang, Y. Tang, W. Xie, X. Zhou, D. Liu, and K. Lu, “VulHawk: Cross-architecture vulnerability detection with entropy-based binary code search,” in *Proceedings of the 30th Annual Network and Distributed System Security Symposium (NDSS)*, 2023.
- [13] Y. Xiao, Z. Xu, W. Zhang, C. Yu, L. Liu, W. Zou, Z. Yuan, Y. Liu, A. Piao, and W. Huo, “Viva: Binary level vulnerability identification via partial signature,” in *Proceedings of the 28th IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*, 2021.
- [14] Z. Liu, S. Liu, L. Pan, C. Chen, E. Ahmed, J. Zhang, and D. Liu, “Vulmatch: Binary-level vulnerability detection through signature,” in *Proceedings of the 18th International Conference on Network and System Security (NSS)*, 2024.
- [15] H. Patel, A. Snit, and M. Polychronakis, “Supply chain reaction: Enhancing the precision of vulnerability triage using code reachability information,” in *Proceedings of the 41st Annual Computer Security Applications Conference (ACSAC)*, 2025.
- [16] H. Zhang and Z. Qian, “Precise and accurate patch presence test for binaries,” in *Proceedings of the 27th USENIX Security Symposium (Security)*, 2018.
- [17] Y. Xu, Z. Xu, B. Chen, F. Song, Y. Liu, and T. Liu, “Patch based vulnerability matching for binary programs,” in *Proceedings of the 29th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, 2020.
- [18] Z. Jiang, Y. Zhang, J. Xu, Q. Wen, Z. Wang, X. Zhang, X. Xing, M. Yang, and Z. Yang, “Pdfff: Semantic-based patch presence testing for downstream kernels,” in *Proceedings of the 27th ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2020.
- [19] S. Yang, Z. Xu, Y. Xiao, Z. Lang, W. Tang, Y. Liu, Z. Shi, H. Li, and L. Sun, “Towards practical binary code similarity detection: Vulnerability verification via patch semantic analysis,” *ACM Transactions on Software Engineering and Methodology (TOSEM)*, 2023.
- [20] Q. Zhan, X. Hu, Z. Li, X. Xia, D. Lo, and S. Li, “PS³: Precise patch presence test based on semantic symbolic signature,” in *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering (ICSE)*, 2024.
- [21] C. Dong, J. Guo, S. Yang, Y. Xiao, Y. Li, H. Li, Z. Li, and L. Sun, “PLocator: Fine-grained patch presence test in binaries via patch code localization,” *ACM Transactions on Software Engineering and Methodology (TOSEM)*, 2026.
- [22] Z. Xu, B. Chen, M. Chandramohan, Y. Liu, and F. Song, “Spain: security patch analysis for binaries towards understanding the pain and pills,” in *Proceedings of the 39th IEEE/ACM International Conference on Software Engineering (ICSE)*, 2017.
- [23] P. Sun, Q. Yan, H. Zhou, and J. Li, “Osprey: A fast and accurate patch presence test framework for binaries,” *Computer Communications*, 2021.
- [24] X. Xu, Q. Zheng, Z. Yan, M. Fan, A. Jia, Z. Zhou, H. Wang, and T. Liu, “Patchdiscovery: Patch presence test for identifying binary vulnerabilities based on key basic blocks,” *IEEE Transactions on Software Engineering (TSE)*, 2023.
- [25] O. Abusabha, J. Uhm, T. Abuhmed, and H. Koo, “A deep dive into function inlining and its security implications for ML-based binary analysis,” in *Proceedings of the 33rd Annual Network and Distributed System Security Symposium (NDSS)*, 2026.
- [26] R. Thomas, “LIEF: Library to instrument executable formats,” 2024. [Online]. Available: <https://lief.re/>
- [27] S. Dinesh, N. Burow, D. Xu, and M. Payer, “RetroWrite: Statically instrumenting COTS binaries for fuzzing and sanitization,” in *Proceedings of the 41st IEEE Symposium on Security and Privacy (SP)*, 2020.

- [28] Y. Li, F. Hossain Shezan, B. Wei, G. Wang, and Y. Tian, “SoK: Towards effective automated vulnerability repair,” in *Proceedings of the 34th USENIX Security Symposium (Security)*, 2025.
- [29] National Security Agency (NSA), “Ghidra: Software reverse engineering (SRE) suite of tools,” 2019. [Online]. Available: <https://ghidra-sre.org/>
- [30] radare2 Project, “radare2: Libre reversing framework for Unix geeks,” <https://github.com/radareorg/radare2>, 2024, accessed: 2026-05.
- [31] E. Bendersky, “pyelftools: A pure-python library for parsing ELF and DWARF,” 2024. [Online]. Available: <https://github.com/eliben/pyelftools>
- [32] Y. Zhang, X. Gao, G. J. Duck, and A. Roychoudhury, “Program vulnerability repair via inductive inference,” in *Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, 2022.
- [33] Y. Hu, Z. Li, K. Shu, S. Guan, D. Zou, S. Xu, B. Yuan, and H. Jin, “SoK: Automated vulnerability repair: Methods, tools, and assessments,” in *Proceedings of the 34th USENIX Security Symposium (Security)*, 2025.
- [34] NIST, “CVE-2017-5225,” 2017. [Online]. Available: <https://nvd.nist.gov/vuln/detail/CVE-2017-5225>
- [35] —, “CVE-2017-15022,” 2017. [Online]. Available: <https://nvd.nist.gov/vuln/detail/CVE-2017-15022>
- [36] —, “CVE-2017-6965,” 2017. [Online]. Available: <https://nvd.nist.gov/vuln/detail/CVE-2017-6965>
- [37] I. U. Haq and J. Caballero, “A survey of binary code similarity,” *ACM Computing Surveys (CSUR)*, 2021.
- [38] A. Qasem, M. Debbabi, B. Lebel, and M. Kassouf, “Binary function clone search in the presence of code obfuscation and optimization over multi-cpu architectures,” in *Proceedings of the 18th ACM Asia Conference on Computer and Communications Security (ASIA CCS)*, 2023.
- [39] S. Eschweiler, K. Yakdan, and E. Gerhards-Padilla, “discovRE: Efficient cross-architecture identification of bugs in binary code,” in *Proceedings of the 23rd Annual Network and Distributed System Security Symposium (NDSS)*, 2016.
- [40] Y. Jia, Z. Yu, and Z. Hong, “Semantic aware-based instruction embedding for binary code similarity detection,” *PLOS ONE*, 2024.
- [41] H. Wang, P. Ma, S. Wang, Q. Tang, S. Nie, and S. Wu, “sem2vec: Semantics-aware assembly tracelet embedding,” *ACM Transactions on Software Engineering and Methodology (TOSEM)*, 2023.
- [42] K. He, Y. Hu, X. Li, Y. Song, Y. Zhao, and D. Gu, “Strtune: Data dependence-based code slicing for binary similarity detection with fine-tuned representation,” *IEEE Transactions on Information Forensics and Security (TIFS)*, 2024.
- [43] W. Zhang, Z. Xu, Y. Xiao, and Y. Xue, “Unleashing the power of pseudo-code for binary code similarity analysis,” *Cybersecurity*, 2022.
- [44] L. Li, L. Song, S. Ding, B. Fung, and P. Charland, “Transforming generic coder LLMs to effective binary code embedding models for similarity detection,” in *Proceedings of the 39th Annual Conference on Neural Information Processing Systems (NeurIPS)*, 2025.
- [45] J. Guo, C. Dong, Y. Ren, S. Li, J. Liu, H. Li, and H. Zhu, “Understanding binary code similarity for real-world vulnerability detection: A large-scale empirical study,” in *Proceedings of the 34th ACM International Conference on the Foundations of Software Engineering (FSE)*, 2026.
- [46] M. Chandramohan, Y. Xue, Z. Xu, Y. Liu, C. Y. Cho, and H. B. K. Tan, “Bingo: Cross-architecture cross-os binary search,” in *Proceedings of the 24th ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE)*, 2016.
- [47] A. Jia, M. Fan, W. Jin, X. Xu, Z. Zhou, Q. Tang, S. Nie, S. Wu, and T. Liu, “1-to-1 or 1-to-n? investigating the effect of function inlining on binary similarity analysis,” *ACM Transactions on Software Engineering and Methodology (TOSEM)*, 2023.
- [48] A. Jia, M. Fan, X. Xu, W. Jin, H. Wang, and T. Liu, “Cross-inlining binary function similarity detection,” in *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering (ICSE)*, 2024.
- [49] DWARF Debugging Information Format Committee, “DWARF debugging information format version 4,” 2010. [Online]. Available: <https://dwarfstd.org/doc/DWARF4.pdf>
- [50] —, “DWARF debugging information format, version 5,” 2017. [Online]. Available: <https://dwarfstd.org/doc/DWARF5.pdf>

TABLE VI: CWE distribution in our vulnerability database.

CWE	CWE Name	# CVEs
CWE-119	Improper Restriction of Operations within the Bounds of a Memory Buffer	17
CWE-122	Heap-based Buffer Overflow	1
CWE-125	Out-of-bounds Read	13
CWE-189	Numeric Errors	2
CWE-190	Integer Overflow or Wraparound	6
CWE-191	Integer Underflow (Wrap or Wraparound)	1
CWE-20	Improper Input Validation	4
CWE-369	Divide By Zero	8
CWE-476	NULL Pointer Dereference	7
CWE-617	Reachable Assertion	1
CWE-787	Out-of-bounds Write	5
CWE-835	Loop with Unreachable Exit Condition (Infinite Loop)	1

APPENDIX

A. Debugging Information and Inlining

DWARF debug information reconstructs source-level function structure in compiled binaries by representing inlined functions using two layers. The first layer is an abstract instance (`DW_TAG_subprogram` with `DW_AT_inline`), which carries the function’s symbolic information, including its name, type, and parameter metadata. The second layer consists of one or more concrete instances (`DW_TAG_inlined_subroutine`), each embedded within the caller’s debugging information entry (DIE) and linked to the corresponding abstract instance via `DW_AT_abstract_origin`. When an inlined body corresponds to a single contiguous instruction sequence, its address coverage is expressed by a (`DW_AT_low_pc`, `DW_AT_high_pc`) pair. However, compiler optimizations such as basic block reordering or cold-path splitting may scatter the inlined instructions across disjoint memory regions. In such cases, DWARF replaces the address pair with a `DW_AT_ranges` attribute that references range lists stored in `.debug_ranges` (DWARF v4 [49]) or `.debug_rnglists` (DWARF v5 [50]) where each holds a list of multiple [low, high) intervals that collectively describe the non-contiguous footprint of the inlined instance. Nested inlining is represented by recursively nested `DW_TAG_inlined_subroutine` DIEs, enabling tools to reconstruct arbitrarily deep call chains by traversing the DIE ancestry and resolving `DW_AT_abstract_origin` at each level. In our experiments, we use DWARF information as ground truth to recover (nested and non-contiguous) inlined regions.

B. Result Visualization

MJ: We design an interactive web interface to visualize individual localization results produced by PinPoint², enabling users to inspect the results for a selected target binary and vulnerable reference. The interface presents the ranked candidate functions, similarity scores, executed stages, and reported ranges. In addition, users can examine the window-level similarity scores for each candidate function together with the DWARF-derived ground-truth vulnerable ranges. Note that the ground truth is displayed only for evaluation and is not used during matching. Figure 7 presents a sample view of the visualization.

²<https://XXXXXXX>

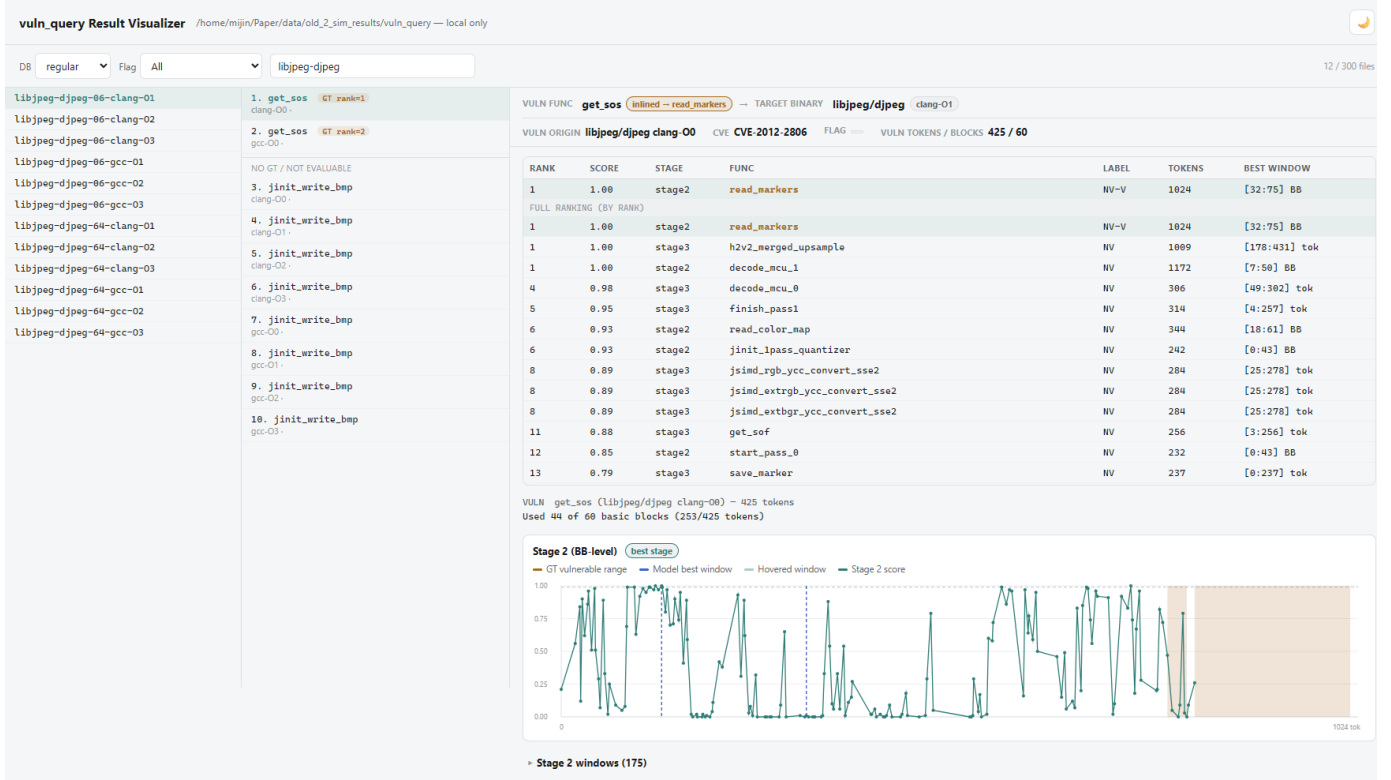


Fig. 7: MJ: Interactive visualization of individual PinPoint localization results. For a selected target binary and vulnerable reference, the interface shows the ranked candidate functions, similarity scores, executed stages, and reported ranges. For the selected candidate function, the lower panel shows the window-level similarity scores together with the DWARF-derived ground-truth vulnerable ranges. The ground truth is displayed only for evaluation and is not used during matching.